

Лабораторная работа №7: Дискретно-событийное моделирование

Мария Улитина

Table of contents

1. Введение	2
2. Модель М/М/с	2
3. Модель М/М/с	2
3.1. Назначение скрипта	3
3.1.1. Что делает скрипт	3
3.1.2. Характеристики модели М/М/с	3
3.2. Параметры модели	3
3.3. Выходные данные	3
3.4. Интерпретация результатов	4
3.4.1. График 1: Распределение времени ожидания	4
3.4.2. График 2: Распределение времени обслуживания	4
3.4.3. График 3: Сходимость среднего времени ожидания	4
3.5. Инициализация проекта DrWatson	4
3.6. Загрузка необходимых пакетов	4
3.7. Задание параметров симуляции	5
3.8. Хранилище для сбора данных	5
3.9. Поведение клиента	5
3.10. Запуск симуляции	6
3.11. Сохранение результатов	6
3.12. Построение графиков	6
3.12.1. График 1: Гистограмма времени ожидания	6
3.12.2. График 2: Гистограмма времени обслуживания	7
3.12.3. График 3: Сходимость среднего	8
3.12.4. Объединение графиков	9
3.13. Вывод статистики в консоль	9

4. Модель Росса	10
5. Модель Росса (резервирование с ремонтом)	10
5.1. Назначение скрипта	10
5.1.1. Что делает скрипт	10
5.1.2. Постановка задачи	11
5.2. Параметры модели	11
5.3. Выходные данные	11
5.4. Теоретическое обоснование	11
5.4.1. Интенсивности переходов	12
5.4.2. Аналитическая формула	12
5.4.3. Условие краха	12
5.5. Инициализация проекта DrWatson	12
5.6. Загрузка необходимых пакетов	12
5.7. Константы модели	12
5.8. Хранилище для истории состояния	13
5.9. Поведение одной машины	13
5.10. Инициализация симуляции	14
5.11. Запуск одного эксперимента	15
5.12. Параметрическое сканирование	15
5.13. Сохранение результатов	16
5.14. График 1: Зависимость времени до краха от S	16
5.15. Аналитическое решение	17
5.16. График 2: Сравнение симуляции с аналитикой	17
5.17. Завершение работы	18

1. Введение

Дискретно-событийное моделирование.

2. Модель M/M/c

3. Модель M/M/c

Авторы: А. В. Королькова, PhD, Кулябов Д. С., DSc

Принадлежность: Российский университет дружбы народов

3.1. Назначение скрипта

Данный скрипт реализует дискретно-событийную симуляцию системы массового обслуживания М/М/с (по классификации Кендалла).

3.1.1. Что делает скрипт

1. Моделирует пуассоновский входящий поток заявок с интенсивностью λ
2. Моделирует экспоненциальное обслуживание с интенсивностью μ на c серверах
3. Собирает статистику о времени ожидания и времени обслуживания
4. Строит графики распределений и сходимости средних
5. Сохраняет результаты в CSV и PNG

3.1.2. Характеристики модели М/М/с

Свойство	Описание
Входящий поток	Пуассоновский (М)
Обслуживание	Экспоненциальное (М)
Число серверов	c (2)
Дисциплина очереди	FIFO

3.2. Параметры модели

Параметр	Значение	Описание
num_customers	100	Количество клиентов
num_servers (c)	2	Число серверов
λ	0.9	Интенсивность входящего потока
μ	0.5	Интенсивность обслуживания (1 сервера)
$\rho = \lambda/(c \cdot \mu)$	0.9	Загрузка системы
seed	123	Зерно генератора случайных чисел

3.3. Выходные данные

Файл	Описание
data/mmc/results.csv	DataFrame с временами прибытия, ожидания, обслуживания
plots/mmc_analysis.png	Составной график (3 панели) --- рис. 7.1

3.4. Интерпретация результатов

3.4.1. График 1: Распределение времени ожидания

Показывает, сколько клиентов ждали в очереди определённое время. При $\rho=0.9$ очередь будет значительной.

3.4.2. График 2: Распределение времени обслуживания

Экспоненциальное распределение с параметром $\mu=0.5$ (среднее = 2).

3.4.3. График 3: Сходимость среднего времени ожидания

Демонстрирует, как накопленное среднее стабилизируется с ростом числа клиентов, что свидетельствует о выходе системы в стационарный режим.

3.5. Инициализация проекта DrWatson

```
using DrWatson
@quickactivate "project"
```

3.6. Загрузка необходимых пакетов

```
using StableRNGs, Distributions, ConcurrentSim, ResumableFunctions
using DataFrames, CSV, Plots
```

3.7. Задание параметров симуляции

Все параметры вынесены в начало для удобства модификации. Используется StableRNG для воспроизводимости результатов.

```
rng = StableRNG(123)
num_customers = 100
num_servers = 2
mu = 1.0 / 2
lam = 0.9
arrival_dist = Exponential(1 / lam)
service_dist = Exponential(1 / mu)
```

```
Exponential{Float64}(θ=2.0)
```

3.8. Хранилище для сбора данных

В массив records будут помещаться кортежи с данными о каждом клиенте.

```
records = []
```

```
Any[]
```

3.9. Поведение клиента

Каждый клиент проходит следующие этапы: 1. Ожидание времени прибытия (timeout) 2. Запрос сервера (request) 3. Ожидание окончания обслуживания (timeout) 4. Освобождение сервера (unlock) 5. Запись временных меток в records

```
@resumable function customer(env, server, id, t_a, d_s)
    @yield timeout(env, t_a)
    t_arr = now(env)
    @yield request(server)
    t_start = now(env)
    @yield timeout(env, rand(rng, d_s))
    t_end = now(env)
    @yield unlock(server)
    push!(records, (id=id, arrival=t_arr, start=t_start, finish=t_end,
                    wait_time=t_start - t_arr, service_time=t_end - t_start))
end
```

```
customer (generic function with 1 method)
```

3.10. Запуск симуляции

Функция создаёт среду, ресурс-сервер, инициализирует клиентов и запускает дискретно-событийную симуляцию.

```
function run_simulation()
    sim = Simulation()
    server = Resource(sim, num_servers)
    arrival_time = 0.0
    for i = 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end
    run(sim)
end

run_simulation()
```

3.11. Сохранение результатов

Преобразование массива записей в DataFrame и сохранение в CSV-файл.

```
df = DataFrame(records)
mkpath(datadir("mmc"))
CSV.write(datadir("mmc", "results.csv"), df)
default(show=true, fmt=:png)
```

3.12. Построение графиков

3.12.1. График 1: Гистограмма времени ожидания

Показывает распределение времени, которое клиенты провели в очереди.

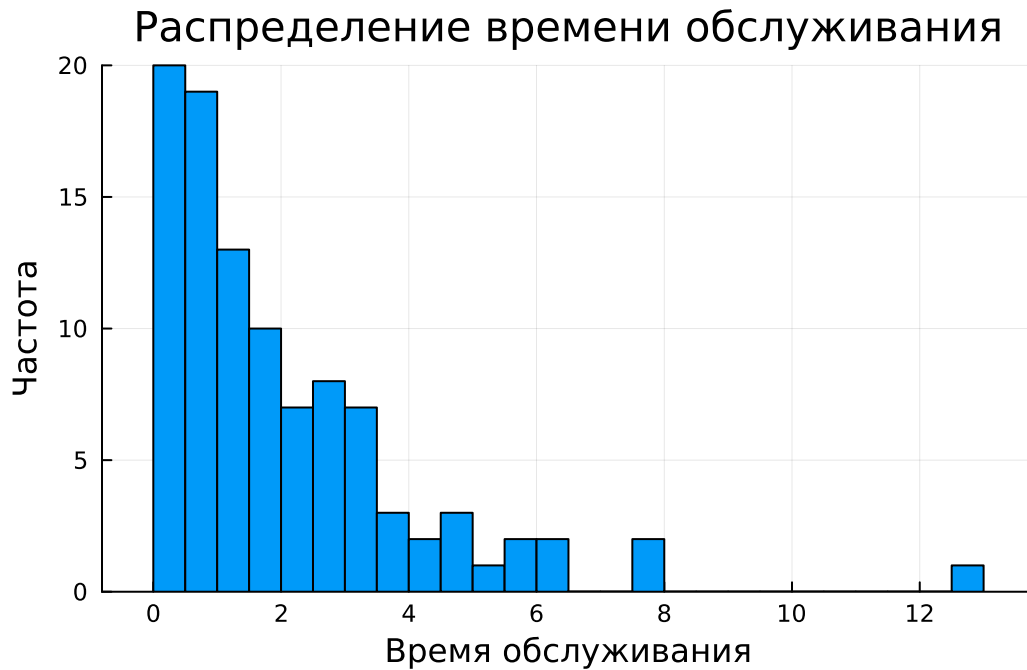
```
p1 = histogram(df.wait_time, bins=30,
               xlabel="Время ожидания", ylabel="Частота",
               title="Распределение времени ожидания в очереди",
               legend=false)
```



3.12.2. График 2: Гистограмма времени обслуживания

Показывает распределение времени обслуживания (экспоненциальное).

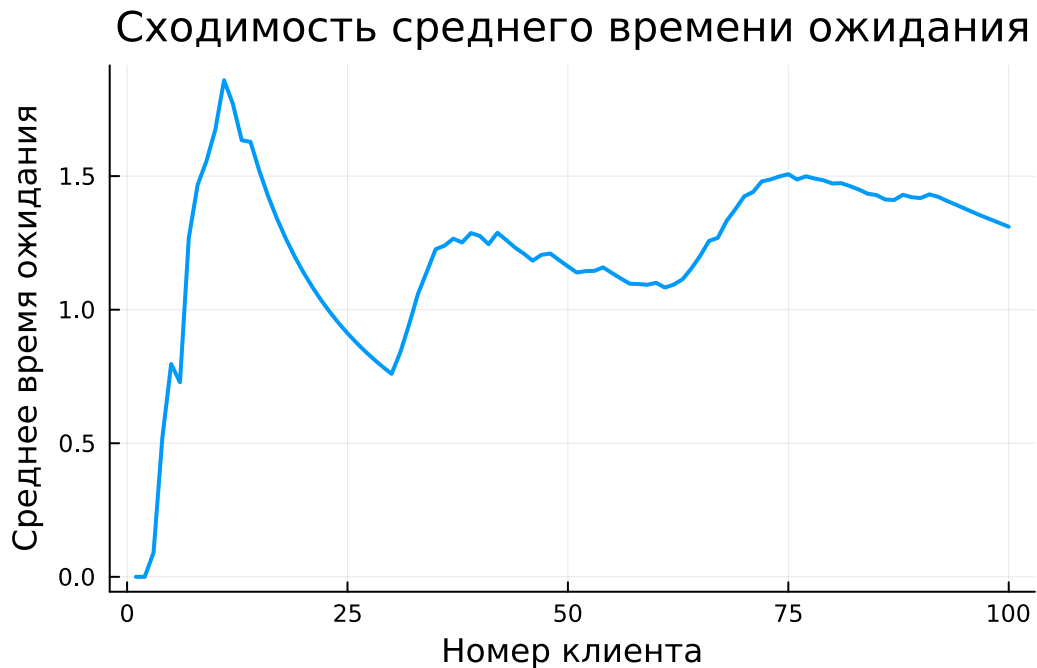
```
p2 = histogram(df.service_time, bins=30,  
               xlabel="Время обслуживания", ylabel="Частота",  
               title="Распределение времени обслуживания",  
               legend=False)
```



3.12.3. График 3: Сходимость среднего

Накопленное среднее время ожидания. Должно выходить на постоянное значение.

```
cumulative_avg = [mean(df.wait_time[1:i]) for i in 1:nrow(df)]  
p3 = plot(1:nrow(df), cumulative_avg,  
          xlabel="Номер клиента", ylabel="Среднее время ожидания",  
          title="Сходимость среднего времени ожидания",  
          legend=false, linewidth=2)
```

3.12.4. Объединение графиков

Все три графика располагаются вертикально (3 строки, 1 столбец).

```
p_final = plot(p1, p2, p3, layout=(3,1), size=(600, 900))
mkpath(plotsdir())
savefig(plotsdir("mmc_analysis.png"))
```

```
"/home/mmilitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab07/project/plots/mmc_analysis.png"
```

3.13. Вывод статистики в консоль

```
println("Среднее время ожидания в очереди: ", round(mean(df.wait_time), digits=3))
println("Среднее время обслуживания: ", round(mean(df.service_time), digits=3))
println("Среднее время в системе: ", round(mean(df.service_time + df.wait_time),
println("Количество обслуженных клиентов: ", nrow(df))
println("\nГрафик сохранён: plots/mmc_analysis.png")
println("Данные сохранены: data/mmc/results.csv")
```

Среднее время ожидания в очереди: 1.311
Среднее время обслуживания: 2.026
Среднее время в системе: 3.337
Количество обслуженных клиентов: 100

График сохранён: plots/mmc_analysis.png
Данные сохранены: data/mmc/results.csv

4. Модель Росса

5. Модель Росса (резервирование с ремонтом)

Авторы: А. В. Королькова, PhD, Кулябов Д. С., DSc

Принадлежность: Российский университет дружбы народов

5.1. Назначение скрипта

Данный скрипт реализует дискретно-событийную симуляцию модели Росса --- системы массового обслуживания с конечной популяцией, резервом и ремонтом.

5.1.1. Что делает скрипт

1. Моделирует работу N идентичных машин, которые постоянно работают
2. S машин находятся в резерве и готовы заменить отказавшие
3. R ремонтников обслуживают сломавшиеся машины
4. При исчерпании резерва система падает (крах)
5. Проводится параметрическое сканирование для разных S и числа ремонтников
6. Строятся графики зависимости времени до краха от параметров
7. Выполняется сравнение с аналитическим решением

Элемент	Описание
---------	----------

5.1.2. Постановка задачи

Элемент	Описание
N	Количество постоянно работающих машин
S	Количество резервных машин
Ремонтники	Обслуживают сломавшиеся машины (1 или 2)
Отказ	Наработка до отказа $\sim \text{Exp}(\lambda)$
Ремонт	Время ремонта $\sim \text{Exp}(\mu)$
Крах	Наступает, когда нет резервной машины для замены

5.2. Параметры модели

Параметр	Значение	Описание
N	10	Основные машины
S_values	[0, 1, 2, 3]	Резервные машины (сканирование)
repair_values	[1, 2]	Количество ремонтников
λ (LAMBDA)	100	Среднее время до отказа (часы)
μ (MU)	1	Среднее время ремонта (часы)
RUNS	2	Количество прогонов для статистики

5.3. Выходные данные

Файл	Описание
data/ross/results.csv	Результаты всех прогонов (S, ремонтники, время краха)
plots/ross_crash_time.png	Зависимость времени до краха от S (лог. шкала)
plots/ross_compare.png	Сравнение симуляции с аналитикой

5.4. Теоретическое обоснование

Система описывается марковским процессом с непрерывным временем. Состояние определяется числом исправных машин i (работающие + резерв).

5.4.1. Интенсивности переходов

- Отказ: $i \rightarrow i-1$ с интенсивностью $\min(N, i) / \lambda$
- Ремонт: $i \rightarrow i+1$ с интенсивностью μ (если есть очередь на ремонт)

5.4.2. Аналитическая формула

Среднее время до краха для одного ремонтника:

$$E[T] = (S + 1) \times (\lambda/N + 1/\mu)$$

5.4.3. Условие краха

Крах наступает при переходе из состояния $i = N$ в $i = N-1$, когда нет резервных машин для замены отказавшей.

5.5. Инициализация проекта DrWatson

```
using DrWatson
@quickactivate "project"
```

5.6. Загрузка необходимых пакетов

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using DataFrames, CSV, Plots
```

5.7. Константы модели

```

const N = 10
const RUNS = 2
const S_values = [0, 1, 2, 3]
const repair_values = [1, 2]
const LAMBDA = 100
const MU = 1
default(show=true, fmt=:png)

```

5.8. Хранилище для истории состояния

```
history_records = []
```

```
Any[]
```

5.9. Поведение одной машины

Каждая машина работает в бесконечном цикле: 1. Работает до отказа (timeout с экспоненциальным распределением) 2. Пытается взять резервную машину из spares 3. Если резерва нет → StopSimulation (крах системы) 4. Если резерв есть → сломанная машина идёт в ремонт 5. После ремонта возвращается в резерв

```

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
        @yield timeout(env, rand(Exponential(LAMBDA)))
        get_spare = take!(spares)
        @yield get_spare | timeout(env)
        if state(get_spare) != ConcurrentSim.idle
            @yield interrupt(value(get_spare))
        else
            throw(StopSimulation("No more spares!"))
        end
    end
end

```

```

        @yield request(repair_facility)
        @yield timeout(env, rand(Exponential(MU)))
        @yield unlock(repair_facility)
        @yield put!(spares, active_process(env))

        busy_repair = repair_facility.level
        queue_len = length(repair_facility.get_queue)
        push!(history_records, (time=now(env),
                                operational=length(spares.items) + N,
                                in_repair=busy_repair,
                                repair_queue=queue_len))
    end
end

```

machine (generic function with 1 method)

5.10. Инициализация симуляции

Создаёт N работающих машин и S резервных.

```

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    S_local::Int,
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S_local
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end
end

```

start_sim (generic function with 1 method)

5.11. Запуск одного эксперимента

Возвращает время до краха и DataFrame с историей состояния.

```
function sim_repair(S_local::Int, num_repair::Int)
    global history_records
    history_records = []

    sim = Simulation()
    repair_facility = Resource(sim, num_repair)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares, S_local)

    try
        run(sim)
    catch e
        if !isa(e, StopSimulation)
            rethrow()
        end
    end

    stop_time = now(sim)
    return stop_time, DataFrame(history_records)
end
```

sim_repair (generic function with 1 method)

5.12. Параметрическое сканирование

Перебираем все комбинации S (резерв) и числа ремонтников. Для каждой комбинации выполняем RUNS прогонов.

```
all_results = []

for S in S_values
    for num_repair in repair_values
        println("S=$S, ремонтников=$num_repair")
        times = []

        for run_num in 1:RUNS
            crash_time, history = sim_repair(S, num_repair)
        end
    end
end
```

```

        push!(times, crash_time)

        if S == 3 && num_repair == 1 && run_num == 1
            global history_1rep = history
        end
        if S == 3 && num_repair == 2 && run_num == 1
            global history_2rep = history
        end
    end

    avg_time = sum(times) / length(times)
    push!(all_results, (S=S, repair_servers=num_repair,
                        avg_crash_time=avg_time, std_crash_time=std(times)))
end
end

```

```

S=0, ремонтников=1
S=0, ремонтников=2
S=1, ремонтников=1
S=1, ремонтников=2
S=2, ремонтников=1
S=2, ремонтников=2
S=3, ремонтников=1
S=3, ремонтников=2

```

5.13. Сохранение результатов

```

df_results = DataFrame(all_results)
mkpath(datadir("ross"))
CSV.write(datadir("ross", "results.csv"), df_results)

```

```

"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab07/project/data/ross/results.csv"

```

5.14. График 1: Зависимость времени до краха от S

Используется логарифмическая шкала по оси Y, так как значения меняются на несколько порядков (от 1 до 25000 часов).


```

p1 = plot(xlabel="S (резервные машины)", ylabel="Среднее время до краха (часы)",
colors = [:red, :blue, :green])
for (idx, r) in enumerate(repair_values)
    sub = filter(row -> row.repair_servers == r, df_results)
    plot!(sub.S, sub.avg_crash_time, marker=:circle, linewidth=2,
        label="$r ремонтник(a)", color=colors[idx])
end
savefig(plotsdir("ross_crash_time.png"))

```

```

└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
└ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194

```

```

"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab07/project/plots/ross_crash_time.png"

```

5.15. Аналитическое решение

Приближённая формула для среднего времени до краха при одном ремонтнике.

```

function analytic_crash_time(N, S, λ, μ)
    return (S + 1) * (λ / N + 1/μ)
end

```

```

analytic_crash_time (generic function with 1 method)

```

5.16. График 2: Сравнение симуляции с аналитикой

```

df_1rep = filter(row -> row.repair_servers == 1, df_results)
analytic_vals = [analytic_crash_time(N, S, LAMBDA, MU) for S in df_1rep.S]

p4 = plot(df_1rep.S, df_1rep.avg_crash_time,
    marker=:circle, label="Симуляция")
plot!(df_1rep.S, analytic_vals, linestyle=:dash, label="Аналитика")
savefig(plotsdir("ross_compare.png"))

```

```

"/home/mmulitina/work/study/2026-1/backup/2026-1-study-simulation-
modeling/labs/lab07/project/plots/ross_compare.png"

```

5.17. Завершение работы

```
println("Готово! Результаты в data/ross/ и plots/")
```

Готово! Результаты в data/ross/ и plots/